

```
In [32]: import numpy as np
import pandas as pd
import os
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from qiskit import transpile
from qiskit.circuit.random import random_circuit
from qiskit_ibm_runtime.fake_provider import FakeKingston
from qiskit_ibm_runtime import SamplerV2
```

```
In [33]: # find all available fake backends on your system
from qiskit_ibm_runtime import fake_provider
import qiskit_ibm_runtime

print("qiskit_ibm_runtime version:", qiskit_ibm_runtime.__version__)
print("\nAvailable fake backends:")
print([x for x in dir(fake_provider) if x.startswith('Fake')])
```

qiskit_ibm_runtime version: 0.47.0

Available fake backends:

```
['FakeAachen', 'FakeAlgiers', 'FakeAlmadenV2', 'FakeArmonkV2', 'FakeAthensV2', 'FakeAuckland', 'FakeBelemV2', 'FakeBerlin', 'FakeBoeblingenV2', 'FakeBogotaV2', 'FakeBoston', 'FakeBrisbane', 'FakeBrooklynV2', 'FakeBrussels', 'FakeBurlingtonV2', 'FakeCairoV2', 'FakeCambridgeV2', 'FakeCasablancaV2', 'FakeCusco', 'FakeEssexV2', 'FakeFez', 'FakeFractionalBackend', 'FakeGeneva', 'FakeGuadalupeV2', 'FakeHanoiV2', 'FakeJakartaV2', 'FakeJohannesburgV2', 'FakeKawasaki', 'FakeKingston', 'FakeKolkataV2', 'FakeKyiv', 'FakeKyoto', 'FakeLagosV2', 'FakeLimaV2', 'FakeLondonV2', 'FakeManhattanV2', 'FakeManilaV2', 'FakeMarrakesh', 'FakeMelbourneV2', 'FakeMiami', 'FakeMontrealV2', 'FakeMumbaiV2', 'FakeNairobiV2', 'FakeNighthawk', 'FakeOsaka', 'FakeOslo', 'FakeOurenseV2', 'FakeParisV2', 'FakePeekskill', 'FakePerth', 'FakePittsburgh', 'FakePoughkeepsieV2', 'FakePrague', 'FakeProviderForBackendV2', 'FakeQuebec', 'FakeQuitoV2', 'FakeRochesterV2', 'FakeRomeV2', 'FakeSantiagoV2', 'FakeSherbrooke', 'FakeSingaporeV2', 'FakeStrasbourg', 'FakeSydneyV2', 'FakeTorino', 'FakeTorontoV2', 'FakeValenciaV2', 'FakeVigoV2', 'FakeWashingtonV2', 'FakeYorktownV2']
```

```
In [34]: # — Initialize parameters —————
backend      = FakeKingston()
sampler      = SamplerV2(mode=backend)

NUM_ITERATIONS = 100
SHOTS          = 4096
MIN_QUBITS     = 2
MAX_QUBITS     = 4
MIN_DEPTH      = 1
MAX_DEPTH      = 6
DIAGRAMS_FOLDER = "circuit_diagrams"

os.makedirs(DIAGRAMS_FOLDER, exist_ok=True)
```

```

print("Backend      :", backend.name)
print("Iterations   :", NUM_ITERATIONS)
print("Shots        :", SHOTS)
print("Qubit range   :", MIN_QUBITS, "→", MAX_QUBITS)
print("Depth range   :", MIN_DEPTH, "→", MAX_DEPTH)
print("Diagrams saved:", DIAGRAMS_FOLDER + "/")

```

```

Backend      : fake_kingston
Iterations   : 100
Shots        : 4096
Qubit range  : 2 → 4
Depth range  : 1 → 6
Diagrams saved: circuit_diagrams/

```

```

In [35]: from qiskit import QuantumCircuit
         from qiskit.circuit.library import (
             HGate, XGate, YGate, ZGate,
             SGate, TGate, SdgGate, TdgGate,
             RZGate, RXGate, RYGate,
             CXGate, CZGate, CYGate,
             SwapGate, ECRGate
         )

         # -----
         # ► DEFINE YOUR GATE ARRAY HERE
         # Add or remove any gates from this list –
         # only these gates will be used in circuit generation
         # -----
         GATE_ARRAY = [
             HGate,      # Hadamard
             XGate,      # Pauli X
             YGate,      # Pauli Y
             ZGate,      # Pauli Z
             SGate,      # S gate
             TGate,      # T gate
             RZGate,     # Rotation Z (parametric)
             RXGate,     # Rotation X (parametric)
             RYGate,     # Rotation Y (parametric)
             CXGate,     # CNOT (2-qubit)
             CZGate,     # Controlled-Z (2-qubit)
             SwapGate,   # SWAP (2-qubit)
         ]

         # -----
         # split into 1Q and 2Q automatically from GATE_ARRAY
         # -----
         def get_num_qubits(gate_cls):
             try:
                 return gate_cls().num_qubits # non-parametric: H, X, CX, S
             except TypeError:
                 return gate_cls(0).num_qubits # parametric: RZ, RX, RY

         def get_gate_name(gate_cls):
             try:
                 return gate_cls().name
             except TypeError:
                 return gate_cls(0).name

```

```
ONE_QUBIT_GATES = [g for g in GATE_ARRAY if get_num_qubits(g) == 1]
TWO_QUBIT_GATES = [g for g in GATE_ARRAY if get_num_qubits(g) == 2]

print("Total gates in GATE_ARRAY :", len(GATE_ARRAY))
print("1-qubit gates :", [get_gate_name(g) for g in ONE_QUBIT_GATES])
print("2-qubit gates :", [get_gate_name(g) for g in TWO_QUBIT_GATES])

# -----
# Helper 1 – generate random circuit using GATE_ARRAY
# -----

def generate_random_circuit(min_q, max_q, min_d, max_d):
    num_qubits = np.random.randint(min_q, max_q + 1)
    depth      = np.random.randint(min_d, max_d + 1)
    seed       = np.random.randint(0, 9999)
    np.random.seed(seed)

    qc = QuantumCircuit(num_qubits)

    for _ in range(depth):
        qubits = list(range(num_qubits))
        np.random.shuffle(qubits)

        i = 0
        while i < len(qubits):
            use_two_qubit = (
                i + 1 < len(qubits)
                and len(TWO_QUBIT_GATES) > 0
                and np.random.random() > 0.4
            )

            if use_two_qubit:
                gate_cls = TWO_QUBIT_GATES[
                    np.random.randint(len(TWO_QUBIT_GATES))
                ]
                # — safely instantiate 2Q gate —————
                try:
                    gate = gate_cls()           # no params needed
                except TypeError:
                    angle = np.random.uniform(0, 2 * np.pi)
                    gate   = gate_cls(angle)    # needs angle
                qc.append(gate, [qubits[i], qubits[i+1]])
                i += 2

            else:
                if len(ONE_QUBIT_GATES) > 0:
                    gate_cls = ONE_QUBIT_GATES[
                        np.random.randint(len(ONE_QUBIT_GATES))
                    ]
                    # — safely instantiate 1Q gate ——
                    try:
                        gate = gate_cls()         # no params needed
                    except TypeError:
                        angle = np.random.uniform(0, 2 * np.pi)
                        gate   = gate_cls(angle)  # needs angle
```

```

        qc.append(gate, [qubits[i]])
        i += 1

    qc.measure_all()
    return qc, num_qubits, depth

# -----
# Helper 2 – extract metadata from counts
# -----
def compute_metadata(counts, shots, qc, transpiled):
    total = sum(counts.values())
    top_bitstring = max(counts, key=counts.get)
    top_prob = counts[top_bitstring] / total
    probs = np.array(list(counts.values())) / total
    entropy = -np.sum(probs * np.log2(probs + 1e-12))
    avg_hamming = sum(
        bs.count('1') * cnt for bs, cnt in counts.items()
    ) / total
    original_ops = dict(qc.count_ops())
    transpiled_ops = dict(transpiled.count_ops())
    total_gates = sum(
        v for k, v in original_ops.items() if k != 'measure'
    )
    two_qubit_count = sum(
        v for k, v in original_ops.items()
        if k in ['cx', 'cz', 'cy', 'swap', 'ecr']
    )
    gate_types = ", ".join(sorted([
        k for k in original_ops if k != 'measure'
    ]))
    return {
        "top_bitstring" : top_bitstring,
        "top_prob"      : round(top_prob, 6),
        "entropy"       : round(entropy, 6),
        "avg_hamming"   : round(avg_hamming, 6),
        "unique_states" : len(counts),
        "gate_types"    : gate_types,
        "total_gates"   : total_gates,
        "two_qubit_gates" : two_qubit_count,
        "original_ops"  : str(original_ops),
        "transpiled_ops" : str(transpiled_ops),
    }

print("\n✓ Helper functions ready.")

```

Total gates in GATE_ARRAY : 12
 1-qubit gates : ['h', 'x', 'y', 'z', 's', 't', 'rz', 'rx', 'ry']
 2-qubit gates : ['cx', 'cz', 'swap']

✓ Helper functions ready.

```

In [37]: # — FOR 100 iterations —————
import os
os.makedirs(DIAGRAMS_FOLDER, exist_ok=True)

QASM_FOLDER = "circuit_qasm" # ← ADD THIS
os.makedirs(QASM_FOLDER, exist_ok=True) # ← ADD THIS

```

```

dataset = []

for i in range(NUM_ITERATIONS):
    print(f"[{i+1:03d}/{NUM_ITERATIONS}]", end=" ")

    # — STEP 1: Generate random circuit —————
    qc, num_qubits, depth = generate_random_circuit(
        MIN_QUBITS, MAX_QUBITS,
        MIN_DEPTH, MAX_DEPTH
    )

    # — STEP 2: Measure all qubits (already in circuit)
    # measure=True in random_circuit handles this

    # — STEP 3: Save circuit diagram —————
    diagram_path = f"{DIAGRAMS_FOLDER}/circuit_{i+1:03d}_q{num_qubits}.png"
    fig = qc.draw('mpl', fold=-1)
    fig.savefig(diagram_path, dpi=100, bbox_inches='tight')
    plt.close(fig)

    # — STEP 3b: Save circuit as QASM file ————— ← ADD FROM HERE
    from qiskit import qasm2
    qasm_string = qasm2.dumps(qc)
    with open(qasm_path, 'w') as f:
        f.write(qasm_string)
    #                                     ← TO HERE

    # — STEP 4: Transpile to backend native gates —————
    transpiled = transpile(
        qc,
        backend = backend,
        optimization_level = 0
    )

    # — STEP 5: Execute (4096 shots) —————
    job = sampler.run([transpiled], shots=SHOTS)
    result = job.result()

    # — STEP 6: Get counts dictionary —————
    counts = result[0].data.meas.get_counts()

    # — STEP 7: Extract metadata —————
    meta = compute_metadata(counts, SHOTS, qc, transpiled)

    # — STEP 8: Convert to row —————
    row = {
        "iteration" : i + 1,
        "num_qubits" : num_qubits,
        "depth" : depth,
        "shots" : SHOTS,
        "unique_states" : meta["unique_states"],
        "top_bitstring" : meta["top_bitstring"],
        "top_prob" : meta["top_prob"],
        "entropy" : meta["entropy"],
        "avg_hamming" : meta["avg_hamming"],
    }

```

```

        "raw_counts"      : str(counts),
        "diagram_path"    : diagram_path,
        "qasm_path"       : qasm_path,      # ← ADD THIS LINE
        "qasm_string"     : qasm_string,    # ← ADD THIS LINE
    }

    # — STEP 9: Append to dataset —————
    dataset.append(row)
    print(
        f"qubits={num_qubits} "
        f"depth={depth} "
        f"unique_states={meta['unique_states']}"
        f"entropy={meta['entropy']:.3f}"
    )

print("\n✓ Loop complete —", len(dataset), "rows collected.")

```

```

[001/100] qubits=3 depth=5 unique_states=8 entropy=2.761
[002/100] qubits=2 depth=3 unique_states=4 entropy=1.027
[003/100] qubits=3 depth=1 unique_states=8 entropy=1.192
[004/100] qubits=4 depth=1 unique_states=12 entropy=1.220
[005/100] qubits=4 depth=2 unique_states=12 entropy=1.197
[006/100] qubits=4 depth=4 unique_states=16 entropy=1.904
[007/100] qubits=3 depth=1 unique_states=7 entropy=1.077
[008/100] qubits=2 depth=2 unique_states=4 entropy=1.036
[009/100] qubits=3 depth=2 unique_states=6 entropy=1.069
[010/100] qubits=3 depth=3 unique_states=8 entropy=1.990
[011/100] qubits=2 depth=3 unique_states=4 entropy=1.019
[012/100] qubits=3 depth=2 unique_states=8 entropy=2.006
[013/100] qubits=4 depth=3 unique_states=13 entropy=1.201
[014/100] qubits=4 depth=2 unique_states=11 entropy=1.182
[015/100] qubits=2 depth=2 unique_states=4 entropy=1.011
[016/100] qubits=2 depth=6 unique_states=4 entropy=1.912
[017/100] qubits=3 depth=1 unique_states=8 entropy=2.010
[018/100] qubits=4 depth=3 unique_states=15 entropy=2.227
[019/100] qubits=3 depth=3 unique_states=8 entropy=1.207
[020/100] qubits=4 depth=4 unique_states=16 entropy=2.889
[021/100] qubits=4 depth=5 unique_states=16 entropy=2.995
[022/100] qubits=4 depth=4 unique_states=11 entropy=1.057
[023/100] qubits=4 depth=5 unique_states=16 entropy=2.751
[024/100] qubits=4 depth=4 unique_states=16 entropy=2.404
[025/100] qubits=4 depth=3 unique_states=14 entropy=1.324
[026/100] qubits=3 depth=2 unique_states=8 entropy=0.926
[027/100] qubits=4 depth=2 unique_states=12 entropy=2.146
[028/100] qubits=2 depth=2 unique_states=4 entropy=1.757
[029/100] qubits=4 depth=1 unique_states=12 entropy=1.749
[030/100] qubits=4 depth=6 unique_states=15 entropy=1.594
[031/100] qubits=2 depth=5 unique_states=4 entropy=1.183
[032/100] qubits=2 depth=2 unique_states=4 entropy=1.013
[033/100] qubits=3 depth=5 unique_states=8 entropy=2.040
[034/100] qubits=4 depth=2 unique_states=13 entropy=1.286
[035/100] qubits=2 depth=5 unique_states=4 entropy=1.838
[036/100] qubits=3 depth=1 unique_states=7 entropy=1.167
[037/100] qubits=3 depth=5 unique_states=7 entropy=1.083
[038/100] qubits=2 depth=5 unique_states=4 entropy=1.211
[039/100] qubits=4 depth=4 unique_states=16 entropy=2.406

```

[040/100]	qubits=2	depth=4	unique_states=4	entropy=0.881
[041/100]	qubits=2	depth=3	unique_states=4	entropy=1.294
[042/100]	qubits=2	depth=3	unique_states=4	entropy=1.174
[043/100]	qubits=2	depth=4	unique_states=4	entropy=1.103
[044/100]	qubits=3	depth=5	unique_states=8	entropy=1.983
[045/100]	qubits=4	depth=4	unique_states=13	entropy=2.226
[046/100]	qubits=2	depth=2	unique_states=4	entropy=1.150
[047/100]	qubits=3	depth=3	unique_states=8	entropy=1.197
[048/100]	qubits=2	depth=1	unique_states=4	entropy=0.961
[049/100]	qubits=2	depth=2	unique_states=4	entropy=1.070
[050/100]	qubits=2	depth=4	unique_states=4	entropy=1.038
[051/100]	qubits=4	depth=2	unique_states=10	entropy=1.000
[052/100]	qubits=3	depth=3	unique_states=6	entropy=0.964
[053/100]	qubits=3	depth=3	unique_states=8	entropy=2.004
[054/100]	qubits=2	depth=5	unique_states=4	entropy=1.988
[055/100]	qubits=3	depth=6	unique_states=8	entropy=0.995
[056/100]	qubits=3	depth=6	unique_states=8	entropy=2.034
[057/100]	qubits=4	depth=3	unique_states=11	entropy=1.272
[058/100]	qubits=3	depth=3	unique_states=8	entropy=2.024
[059/100]	qubits=2	depth=1	unique_states=4	entropy=1.018
[060/100]	qubits=4	depth=6	unique_states=16	entropy=2.267
[061/100]	qubits=3	depth=2	unique_states=6	entropy=1.184
[062/100]	qubits=3	depth=1	unique_states=6	entropy=1.078
[063/100]	qubits=3	depth=1	unique_states=8	entropy=1.104
[064/100]	qubits=2	depth=2	unique_states=4	entropy=1.009
[065/100]	qubits=4	depth=6	unique_states=16	entropy=1.884
[066/100]	qubits=3	depth=2	unique_states=7	entropy=1.145
[067/100]	qubits=4	depth=5	unique_states=16	entropy=2.852
[068/100]	qubits=2	depth=1	unique_states=4	entropy=1.798
[069/100]	qubits=4	depth=5	unique_states=16	entropy=2.263
[070/100]	qubits=2	depth=3	unique_states=4	entropy=1.074
[071/100]	qubits=2	depth=4	unique_states=4	entropy=0.822
[072/100]	qubits=3	depth=2	unique_states=7	entropy=1.160
[073/100]	qubits=3	depth=2	unique_states=8	entropy=1.564
[074/100]	qubits=3	depth=4	unique_states=8	entropy=1.731
[075/100]	qubits=2	depth=4	unique_states=4	entropy=1.657
[076/100]	qubits=2	depth=1	unique_states=4	entropy=1.120
[077/100]	qubits=3	depth=3	unique_states=7	entropy=1.169
[078/100]	qubits=2	depth=2	unique_states=4	entropy=0.998
[079/100]	qubits=3	depth=3	unique_states=8	entropy=1.799
[080/100]	qubits=4	depth=2	unique_states=16	entropy=2.234
[081/100]	qubits=2	depth=5	unique_states=4	entropy=1.846
[082/100]	qubits=2	depth=1	unique_states=4	entropy=1.006
[083/100]	qubits=2	depth=1	unique_states=4	entropy=1.075
[084/100]	qubits=3	depth=1	unique_states=6	entropy=1.093
[085/100]	qubits=2	depth=5	unique_states=4	entropy=1.144
[086/100]	qubits=3	depth=5	unique_states=8	entropy=1.904
[087/100]	qubits=3	depth=5	unique_states=8	entropy=2.229
[088/100]	qubits=3	depth=6	unique_states=8	entropy=1.710
[089/100]	qubits=4	depth=2	unique_states=11	entropy=1.245
[090/100]	qubits=3	depth=5	unique_states=8	entropy=1.494
[091/100]	qubits=4	depth=1	unique_states=10	entropy=1.130
[092/100]	qubits=2	depth=4	unique_states=4	entropy=1.035
[093/100]	qubits=4	depth=6	unique_states=15	entropy=2.083
[094/100]	qubits=3	depth=3	unique_states=8	entropy=1.966
[095/100]	qubits=2	depth=4	unique_states=4	entropy=0.972

```
[096/100] qubits=2  depth=1  unique_states=4  entropy=0.991
[097/100] qubits=3  depth=1  unique_states=7  entropy=1.093
[098/100] qubits=4  depth=2  unique_states=12  entropy=1.338
[099/100] qubits=4  depth=4  unique_states=16  entropy=1.556
[100/100] qubits=3  depth=5  unique_states=8  entropy=2.100
```

✓ Loop complete – 100 rows collected.

```
In [38]: # — Construct DataFrame —————
df = pd.DataFrame(dataset)

print("Shape      :", df.shape)
print("Columns    :", list(df.columns))
print("\nFirst 5 rows:")
df.head()
```

```
Shape      : (100, 13)
Columns    : ['iteration', 'num_qubits', 'depth', 'shots', 'unique_
states', 'top_bitstring', 'top_prob', 'entropy', 'avg_hamming', 'raw
_counts', 'diagram_path', 'qasm_path', 'qasm_string']
```

First 5 rows:

```
Out[38]:
```

	iteration	num_qubits	depth	shots	unique_states	top_bitstring	top_pr
0	1	3	5	4096	8	010	0.2155
1	2	2	3	4096	4	00	0.6486
2	3	3	1	4096	8	010	0.6386
3	4	4	1	4096	12	0000	0.6501
4	5	4	2	4096	12	0000	0.6367

```
In [39]: # — Sanity check before export —————
print("Null values:\n", df.isnull().sum())
print("\nNumeric summary:")
print(df[["num_qubits", "depth",
```



```

        "unique_states", "top_prob",
        "entropy", "avg_hamming"
    ]].describe().round(4))

```

Null values:

```

iteration      0
num_qubits    0
depth         0
shots         0
unique_states 0
top_bitstring 0
top_prob      0
entropy       0
avg_hamming   0
raw_counts    0
diagram_path  0
qasm_path     0
qasm_string   0
dtype: int64

```

Numeric summary:

	num_qubits	depth	unique_states	top_prob	entropy	avg_hamming
count	100.000	100.0000	100.000	100.0000	100.0000	1
mean	2.960	3.1600	8.210	0.5385	1.5136	
std	0.803	1.5937	4.164	0.1538	0.5392	
min	2.000	1.0000	4.000	0.1746	0.8216	
25%	2.000	2.0000	4.000	0.3862	1.0769	
50%	3.000	3.0000	8.000	0.5944	1.2155	
75%	4.000	5.0000	11.000	0.6478	1.9705	
max	4.000	6.0000	16.000	0.7788	2.9954	

```

In [40]: # — Export to CSV —————
csv_path = "benchmark.csv"
df.to_csv(csv_path, index=False)

print(f"✓ Saved '{csv_path}'")
print(f"  Rows    : {len(df)}")
print(f"  Columns : {len(df.columns)}")
print(f"  Size     : {os.path.getsize(csv_path) / 1024:.1f} KB")

```

```

✓ Saved 'benchmark.csv'
  Rows    : 100
  Columns : 13
  Size     : 46.7 KB

```